

step02_03_ETL_cleaning

October 6, 2026

```
[11]: import duckdb
import json
import pandas as pd
from pathlib import Path

pd.set_option("display.width", 220)
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_rows", 120)

BASE_DIR = Path.cwd().parent
DATA_DIR = BASE_DIR / "data_sets"
RESULT_DIR = BASE_DIR / "results"
RESULT_DIR.mkdir(exist_ok=True)
TMP_DIR = RESULT_DIR / "duckdb_tmp"
TMP_DIR.mkdir(exist_ok=True)

# Updated to create/connect to the new database in the results directory
con = duckdb.connect(str(RESULT_DIR / "step02_03_ETL_cleaning.duckdb"))
con.execute("SET memory_limit='6GB'") # lower this if your laptop has < 8 GB
→RAM
con.execute(f"SET temp_directory='{TMP_DIR.as_posix()}'"")
con.execute("SET preserve_insertion_order=false")

FINDINGS = {} # everything that matters is collected here -> saved as JSON at
→the end

def q(sql):
    return con.execute(sql).df()

def save(df, name):
    df.to_csv(RESULT_DIR / name, index=False)
    print(f" -> saved results/{name}")

[12]: # ## 0. Load tables into DuckDB (delay is loaded once and cached in the .duckdb
→file)
def load(name, fname, types=None):
    # Updated to load from RESULT_DIR instead of DATA_DIR
```

```

path = (RESULT_DIR / fname).as_posix()
t = f", types={types}" if types else ""
con.execute(f"CREATE OR REPLACE TABLE {name} AS "
            f"SELECT * FROM read_csv('{path}', header=true {t})")

# Updated filenames to target the imputed/resolved datasets
load("train_details", "dim_train_resolved.csv", {"train_no": "BIGINT"})
load("station_full_names", "dim_station_resolved.csv")
load("schedule", "combined_schedule_imputed.csv", {"train_no": "BIGINT",
    ↪ "station_no": "INTEGER"})

already = con.execute("SELECT count(*) FROM information_schema.tables WHERE
    ↪ table_name='delay'").fetchone()[0]
if not already:
    print("Loading combined_delay_imputed.csv (one-time, a few minutes)...")
    # Updated to target combined_delay_imputed.csv
    load("delay", "combined_delay_imputed.csv", {"date": "DATE", "station_no":
    ↪ "INTEGER", "delay": "DOUBLE", "train_no": "BIGINT"})

for t in ["train_details", "station_full_names", "schedule", "delay"]:
    print(t, con.execute(f"SELECT count(*) FROM {t}").fetchone()[0])

# Helper tables used repeatedly
con.execute("CREATE OR REPLACE TABLE sched_stations AS SELECT DISTINCT
    ↪ station_name FROM schedule")
con.execute("CREATE OR REPLACE TABLE delay_stations AS SELECT DISTINCT
    ↪ station_name FROM delay")

```

```

train_details 8720
station_full_names 9027
schedule 172112
delay 37832789

```

[12]: <_duckdb.DuckDBPyConnection at 0x213862395b0>

```

[13]: # The query to check for negative delays
early_arrival_query = """
SELECT
    COUNT(*) AS total_early_trains,
    MIN(delay) AS max_early_minutes,
    AVG(delay) AS average_early_minutes
FROM delay
WHERE delay < 0
"""

print("--- EARLY ARRIVAL (NEGATIVE DELAY) METRICS ---")

```

```
# Execute using the q() helper function defined in your setup cell
results = q(early_arrival_query)
print(results.to_string(index=False))
```

--- EARLY ARRIVAL (NEGATIVE DELAY) METRICS ---

total_early_trains	max_early_minutes	average_early_minutes
850395	-120.0	-13.53392

```
[14]: # =====
# POSITIVE DELAY OUTLIER METRICS
# =====
outlier_query = """
SELECT
    MAX(delay) AS max_positive_delay,
    AVG(delay) AS mean_delay,
    STDDEV(delay) AS std_dev_delay,
    -- Checking for a 24-hour (1440 mins) cutoff as an initial domain threshold
    COUNT(CASE WHEN delay > 1440 THEN 1 END) as delays_over_24_hours
FROM delay
"""

print("--- POSITIVE DELAY OUTLIER METRICS ---")

outlier_metrics = q(outlier_query)
print(outlier_metrics.to_string(index=False))

# Safely store the extracted statistics in your global FINDINGS dictionary
FINDINGS["positive_outlier_metrics"] = {
    "max_positive_delay": float(outlier_metrics["max_positive_delay"][0]),
    "mean_delay": float(outlier_metrics["mean_delay"][0]),
    "std_dev_delay": float(outlier_metrics["std_dev_delay"][0]),
    "delays_over_24_hours": int(outlier_metrics["delays_over_24_hours"][0])
}
```

--- POSITIVE DELAY OUTLIER METRICS ---

max_positive_delay	mean_delay	std_dev_delay	delays_over_24_hours
525592.0	36.855588	284.151586	15564

```
[15]: outlier_script = """
-- Step 1: Create a clean working table by dropping transmission noise (delays_
    ↳ 1440 mins)
CREATE OR REPLACE TABLE delay_cleaned AS
SELECT * FROM delay
WHERE delay <= 1440;

-- Step 2: Calculate the true statistical boundaries for genuine bottlenecks
WITH GlobalStats AS (
    SELECT
```

```

        AVG(delay) AS true_mean,
        STDDEV(delay) AS true_std_dev
    FROM delay_cleaned
)
SELECT
    true_mean,
    true_std_dev,
    (true_mean + (2 * true_std_dev)) AS outlier_threshold_2sd,
    (true_mean + (3 * true_std_dev)) AS outlier_threshold_3sd,
    (SELECT COUNT(*) FROM delay_cleaned WHERE delay > (true_mean + (2 *
↳true_std_dev))) AS total_bottleneck_records
FROM GlobalStats;
"""

print("--- TRUE BOTTLENECK BOUNDARIES (POST-NOISE REMOVAL) ---")
# Using the q() helper function defined in your notebook setup
results = q(outlier_script)
print(results.to_string(index=False))

```

--- TRUE BOTTLENECK BOUNDARIES (POST-NOISE REMOVAL) ---

```

FloatProgress(value=0.0, layout=Layout(width='auto'),
↳style=ProgressStyle(bar_color='black'))

true_mean  true_std_dev  outlier_threshold_2sd  outlier_threshold_3sd
total_bottleneck_records
35.647028      66.907683              169.462393              236.370076
1311727

```

```

[16]: output_filename = "combined_delay_cleaned.csv"
output_path = RESULT_DIR / output_filename

# Retrieve the final row count for verification
final_rows = con.execute("SELECT COUNT(*) FROM delay_cleaned").fetchone()[0]

print(f"Exporting {final_rows:,} cleaned records to CSV. This will take a
↳moment...")

# Execute the out-of-core export
con.execute(f"COPY delay_cleaned TO '{output_path.as_posix()}' (HEADER,
↳DELIMITER ',')")

print(f" -> Successfully saved to results/{output_filename}")

```

Exporting 37,817,225 cleaned records to CSV. This will take a moment...

```

FloatProgress(value=0.0, layout=Layout(width='auto'),
↳style=ProgressStyle(bar_color='black'))

-> Successfully saved to results/combined_delay_cleaned.csv

```

```
[19]: dim_date_script = """
CREATE OR REPLACE TABLE dim_date AS
SELECT DISTINCT
    date AS date_id,
    EXTRACT('year' FROM date) AS year,
    EXTRACT('quarter' FROM date) AS quarter,
    EXTRACT('month' FROM date) AS month,
    MONTHNAME(date) AS month_name,
    EXTRACT('day' FROM date) AS day_of_month,
    DAYNAME(date) AS day_name,
    EXTRACT('isodow' FROM date) AS day_of_week, -- 1 = Monday, 7 = Sunday
    CASE WHEN EXTRACT('isodow' FROM date) IN (6, 7) THEN TRUE ELSE FALSE END AS is_weekend
FROM delay_cleaned
WHERE date IS NOT NULL
ORDER BY date;
"""

print("Generating Concept Hierarchy (Time Dimension)...")
# Execute using your q() helper function
q(dim_date_script)

# Verify the hierarchy generation
check_dim_date = "SELECT * FROM dim_date LIMIT 5;"
print(q(check_dim_date).to_string(index=False), "\n(truncated...)")

output_filename = "dim_date.csv"
output_path = RESULT_DIR / output_filename

# Retrieve the row count for verification
date_rows = con.execute("SELECT COUNT(*) FROM dim_date").fetchone()[0]

print(f"Exporting {date_rows:,} unique dates as a concept hierarchy...")

# Execute the out-of-core export
con.execute(f"COPY dim_date TO '{output_path.as_posix()}' (HEADER, DELIMITER_,
↳ ',')")
print(f" -> Successfully saved to results/{output_filename}")
```

Generating Concept Hierarchy (Time Dimension)...

date_id	year	quarter	month	month_name	day_of_month	day_name	day_of_week
2025-02-08	2025	1	2	February	8	Saturday	6
2025-02-09	2025	1	2	February	9	Sunday	7
2025-02-10	2025	1	2	February	10	Monday	1

is_weekend
True
True
False

2025-02-11	2025	1	2	February	11	Tuesday	2
------------	------	---	---	----------	----	---------	---

False

2025-02-12	2025	1	2	February	12	Wednesday	3
------------	------	---	---	----------	----	-----------	---

False

(truncated...)

Exporting 365 unique dates as a concept hierarchy...

-> Successfully saved to results/dim_date.csv

```
[20]: # =====
# APPEND REMAINING FINDINGS & EXPORT JSON
# =====

# 1. Store Early Arrival Findings
# (Re-querying briefly to ensure accuracy since the 'results' variable was
↳ overwritten in Cell 5)
early_df = q("SELECT COUNT(*) AS total_early_trains, MIN(delay) AS
↳ max_early_minutes, AVG(delay) AS average_early_minutes FROM delay WHERE
↳ delay < 0")

FINDINGS["early_arrival_validation"] = {
    "max_early_minutes": float(early_df["max_early_minutes"][0]),
    "average_early_minutes": float(early_df["average_early_minutes"][0]),
    "conclusion": "Negative values represent valid operational early arrivals.
↳ Retained intact as negative deltas without mathematical smoothing."
}

# 2. Store True Bottleneck Boundaries
# (Extracting directly from the 'results' dataframe generated in your Cell 5)
FINDINGS["bottleneck_thresholds"] = {
    "true_mean": float(results["true_mean"][0]),
    "true_std_dev": float(results["true_std_dev"][0]),
    "high_risk_2sd": float(results["outlier_threshold_2sd"][0]),
    "critical_risk_3sd": float(results["outlier_threshold_3sd"][0]),
    "total_bottleneck_records": int(results["total_bottleneck_records"][0]),
    "conclusion": "High-risk structural bottlenecks mathematically defined as
↳ any continuous delay exceeding 169.46 minutes."
}

# 3. Store Concept Hierarchy details
# (Utilizing the 'date_rows' variable generated in your Cell 7)
FINDINGS["concept_hierarchy"] = {
    "table_created": "dim_date",
    "total_unique_dates": int(date_rows),
    "extracted_attributes": [
        "year", "quarter", "month", "month_name",
        "day_of_month", "day_name", "day_of_week", "is_weekend"
    ],
}
```

```

        "conclusion": "Raw temporal data successfully transformed into a
↳ multidimensional concept hierarchy for OLAP drilling."
    }

# 4. Save the fully populated FINDINGS dictionary to JSON
etl_findings_path = RESULT_DIR / "step02_03_ETL_FINDINGS.json"
with open(etl_findings_path, "w") as f:
    json.dump(FINDINGS, f, indent=4)

print("--- FINAL ETL FINDINGS (OUTLIERS & TEMPORAL) ---")
print(json.dumps(FINDINGS, indent=2))
print(f"\n-> Successfully saved complete findings to {etl_findings_path.name}")

```

```

--- FINAL ETL FINDINGS (OUTLIERS & TEMPORAL) ---
{
  "positive_outlier_metrics": {
    "max_positive_delay": 525592.0,
    "mean_delay": 36.855588085245316,
    "std_dev_delay": 284.1515855510185,
    "delays_over_24_hours": 15564
  },
  "early_arrival_validation": {
    "max_early_minutes": -120.0,
    "average_early_minutes": -13.533920119473892,
    "conclusion": "Negative values represent valid operational early arrivals.
Retained intact as negative deltas without mathematical smoothing."
  },
  "bottleneck_thresholds": {
    "true_mean": 35.64702773669935,
    "true_std_dev": 66.90768268196882,
    "high_risk_2sd": 169.462393100637,
    "critical_risk_3sd": 236.37007578260582,
    "total_bottleneck_records": 1311727,
    "conclusion": "High-risk structural bottlenecks mathematically defined as
any continuous delay exceeding 169.46 minutes."
  },
  "concept_hierarchy": {
    "table_created": "dim_date",
    "total_unique_dates": 365,
    "extracted_attributes": [
      "year",
      "quarter",
      "month",
      "month_name",
      "day_of_month",
      "day_name",
      "day_of_week",
      "is_weekend"
    ]
  }
}

```

```
    ],  
    "conclusion": "Raw temporal data successfully transformed into a  
multidimensional concept hierarchy for OLAP drilling."  
  }  
}
```

-> Successfully saved complete findings to step02_03_ETL_FINDINGS.json

[]: